HW4 P1 Bootcamp

Autoregressive Language Modeling with a Causal Transformer Decoder

TAs: Manigandan, Yixiong, Olivier, Sid

Agenda for today

- Logistics
- Homework Objectives
- Setup & Workflow
- Introduction of concepts covered
- Task 1
- Task 2

Important Dates

OUT: **March 27**, 2026, 6:00 PM ET

Early Bonus deadline: **April 10**, 2026, 11:59 PM ET

DUE: **April 24**, 2026, 11:59 PM ET

WARNING: This assignment is designed to be significantly more challenging than previous assignments. We recommend starting early and reaching out to Piazza if you get stuck.

Materials

1. Writeup (PDF)
2. Main Handout → IDL-HW4.zip file
3. Starter Notebook → HWP1_nb.ipynb

Materials

1. Writeup PDF
2. **IDL-HW4 .zip file (Common for both Part 1 and Part 2 for this HW)**
3. Starter Notebook .ipynb

Homework Objectives

- Implement **self-attention** from scratch, including **scaled dot-product attention** and **multi-head attention**.
- Understand the fundamentals of Language Modeling, including **next-token prediction** and **full-sequence generation**.
- Implement **positional encoding** to incorporate sequence order in transformer models.
- Develop and apply **causal and padding masks** to enforce autoregressive behavior and handle variable-length inputs.
- Explore multihead attention and its ability to capture **multiple dependencies in the input sequence**.
- Implement key transformer components such as **self-attention layers, feedforward layers, and full decoder layers**.
- **Train** a **causal transformer decoder model for autoregressive text generation**.
- **Generate** text samples using the trained language model to evaluate its qualitative performance.

Setup & Workflow

Test Suite Integration: Your code will be tested in a manner similar to HWP1's.

Local Development: You will be able to perform most of your initial development locally, reducing the need for compute resources.

Hands-on Experience: This assignment provides an opportunity to build an end-to-end deep learning pipeline from scratch. We will be substantially reducing the number of abstractions compared to previous assignments.

The Handout (IDL-HW4 .zip file)

Dataset, main hw4 library, MyTorch, Tests

```
handout_directory/
├── hw4_data_subset/
├── hw4lib/
├── mytorch/
├── tests/
├── README.md
└── requirements.txt
```

For our provided notebooks to work, your notebook's current working directory must be the **same as the handout directory**

The Handout: Dataset

You will be able to test your code locally using the subset in the handout

```
hw4_data_subset/
├── hw4p1_data/ ...............
│   ├── test/
│   ├── train/
│   └── valid/
└── hw4p2_data/ ...............
    ├── dev-clean/
    │   ├── fbank/
    │   └── text/
    ├── test-clean/
    │   └── fbank/
    └── train-clean-100/
        ├── fbank/
        └── text/
```

The Handout: The Main Library (hw4lib)

You will incrementally implement components of hw4lib to build and train two models:

HW4P1: A Decoder-only Transformer for causal language modeling

HW4P2: An Encoder-Decoder Transformer for end-to-end speech recognition

```
hw4lib/
├── data/
│   ├── asr_dataset.py
│   ├── lm_dataset.py
│   ├── tokenizer.py
│   └── tokenizer_jsons/
├── decoding/
│   └── sequence_generator.py
├── model/
│   ├── decoder_layers.py
│   ├── encoder_layers.py
│   ├── masks.py
│   ├── positional_encoding.py
│   ├── speech_embedding.py
│   ├── sublayers.py
│   └── transformers.py
├── trainers/
│   ├── asr_trainer.py
│   ├── base_trainer.py
│   └── lm_trainer.py
└── utils/
    ├── create_lr_scheduler.py
    └── create_optimizer.py
```

The Handout: MyTorch

You will implement **multi-head attention** and **scaled dot product attention** here and add it to your mytorch collection :-)

```
mytorch/
└── nn/
    ├── activation.py
    ├── linear.py
    ├── multi_head_attention.py
    └── scaled_dot_product_attention.py
```

The Handout: Tests

You will be provided with a test suite to verify your implementation.

```
tests/
├── test_dataset*.py
├── test_decoding.py
├── test_decoderlayers*.py
├── test_encoderlayers*.py
├── test_hw4p1.py
├── test_mask*.py
├── test_mytorch*.py
├── test_positional_encoding.py
├── test_sublayers*.py
├── test_transformers*.py
└── testing_framework.py
```

Please read the writeup **(page 7)** instructions carefully for different ways of maintaining your handout so it remains consistent for both parts of the homework

Language Modeling

In this homework, we deal with modelling the **probability distribution of symbol sequences** in a language, and generating sequences from this distribution.

$$P(x_0, x_1, \ldots)$$

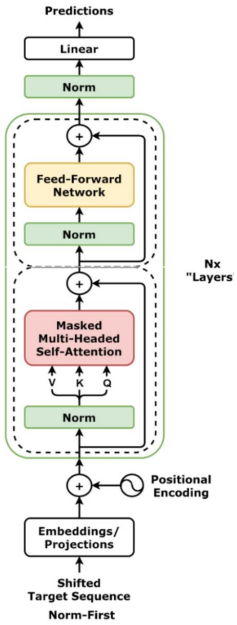
How do we define **language**?

a language is defined as the set of all valid sequences of symbols from a vocabulary

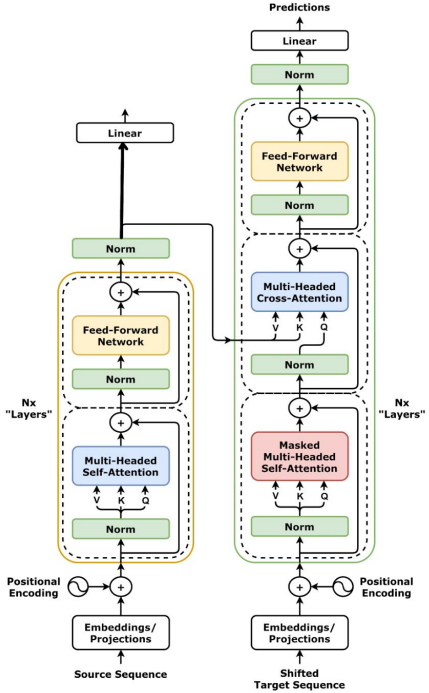
$$P(x_0, x_1, \ldots) = P(x_0)\, P(x_1 \mid x_0)\, P(x_2 \mid x_0, x_1) \cdots = \prod_t P(x_t \mid x_0, \ldots, x_{t-1}).$$

Please read the Preamble section (page 8) before starting this homework

Decoder-Only (hw4p1) vs Encoder-Decoder Transformer (hw4p2)



(a) Decoder-Only Transformer
(GPT-style, HW4P1)



(b) Encoder-Decoder Transformer
(Original Transformer, HW4P2)

Decoder-only transformer

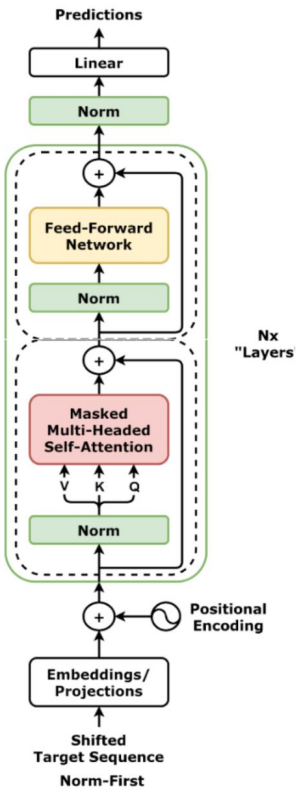
The architecture follows the design of GPT-2, a foundation for modern language models such as GPT-4, GPT-4o, and LLaMA-3

Use cases: Language modeling, text generation, completion tasks

We aim to train a **decoder-only transformer** to model the probability of a sequence of tokens autoregressively:

$$P(x_1, ..., x_N) = \prod_{t=1}^{N} P(x_t | x_1, ..., x_{t-1}) \tag{1}$$

where $x_1, ..., x_N$ represents a sequence of tokens and each conditional probability $P(x_t | x_1, ..., x_{t-1})$ is modeled using our transformer.



(a) Decoder-Only Transformer (GPT-style, HW4P1)

Training

We use self-supervision: every token predicts the next token. So the model learns next-token prediction without manual labels.

Input : <SOS> **I** swam across the river

Target: **I** swam across the river <EOS>

Token Embedding

Each token in the input sequence is converted into a sequence of **D-dimensional** embedding vectors using the model's embedding layer:

$$x_1, ..., x_N \in \{1, ..., K\} \mapsto E(x_1), ..., E(x_N) \in \mathbb{R}^D$$

Implemented using **PyTorch's nn.Embedding** which maps token indices to dense vectors.

The D-dimensional embedding vectors are injected with positional encodings and then passed through the decoder layers

Output

The model produces a sequence of hidden states **h1,...,hN,** each of dimensionality D. To obtain **logits** over the vocabulary (K), we apply a final linear transformation:

$$Y = XW^{(p)}$$

- $W^{(p)} \in \mathbb{R}^{D \times K}$ is the **projection matrix**.
- $X \in \mathbb{R}^{N \times D}$ is the output of the hidden states of the model.
- $Y \in \mathbb{R}^{N \times K}$ are **logits**, representing unnormalized scores over the vocabulary.

Softmax and Cross-Entropy Loss

We can apply the softmax function to obtain probabilities:

$$P(x_t|x_1, ..., x_{t-1}) = \text{softmax}(Y_t)$$

However, we **do not** explicitly compute softmax in the code. Instead, PyTorch's nn.CrossEntropyLoss:

- Takes the raw logits Y and the ground-truth token sequence.
- Internally applies log-softmax and NLL before computing the loss (numerically stable).

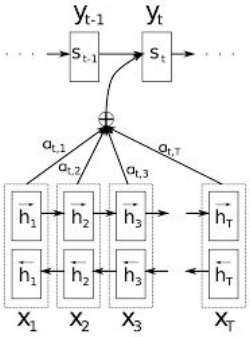
Generation: Sampling Strategies

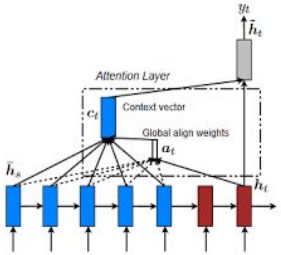
Once, trained, your model can generate sequences and then you will implement greedy search or one of the mentioned sampling strategies:

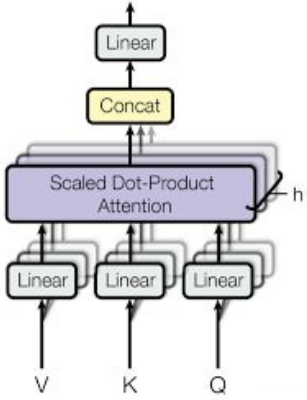
- Beam Search
- Top-K Sampling
- Nucleus Sampling (Top-p Sampling)
- Temperature Sampling

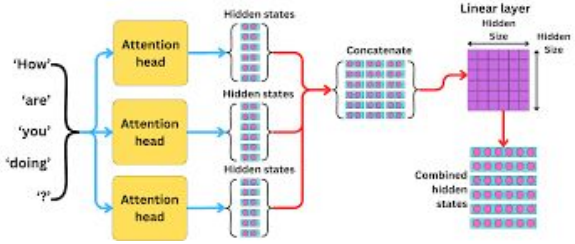
Attention Evolution

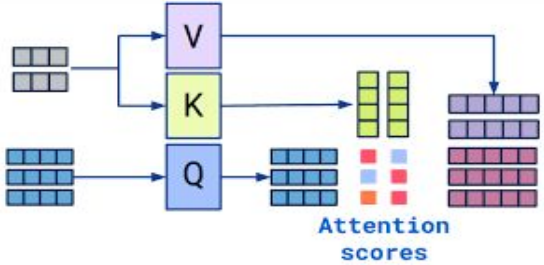
- **Bahdanau Attention (2014)**
- **Luong Attention (2015)**
- **QKV Attention ⇔ Transformers (2017)**
- **Multi Head Attention (HW 4)**
- **Self Attention**
- **Cross Attention**

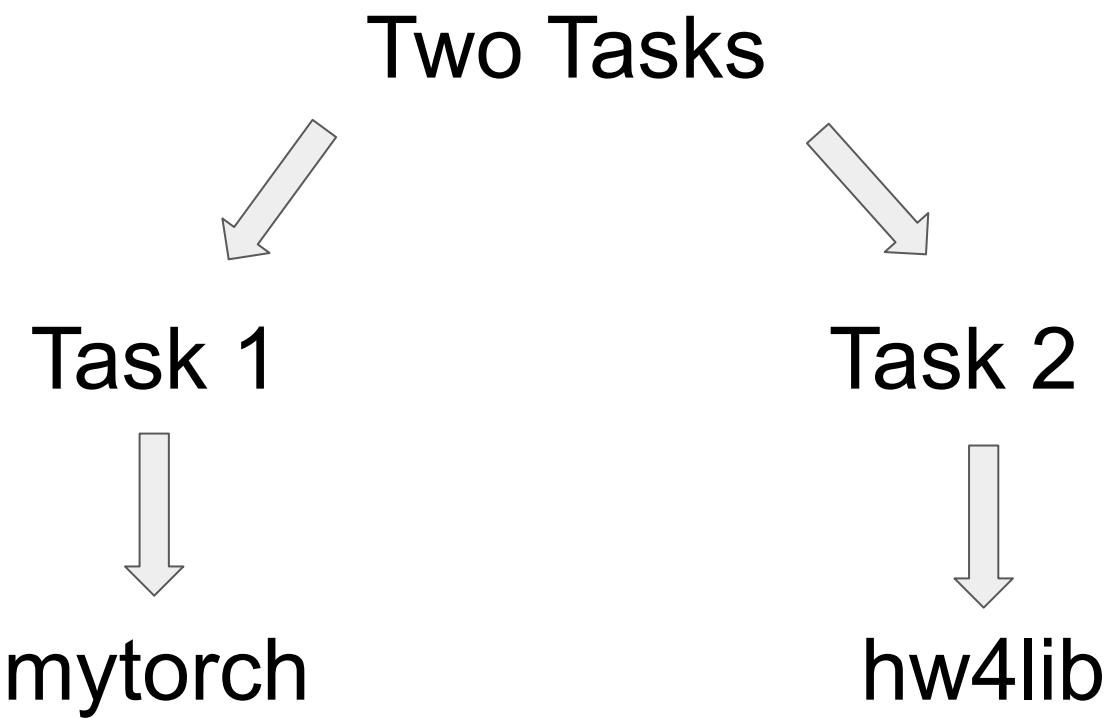











Two Tasks
Task 1
Task 2
mytorch
hw4lib

Task 1:

Goal:

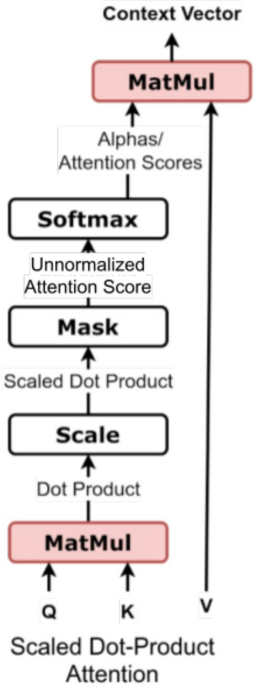


Figure 2: Scaled Dot-Product Attention

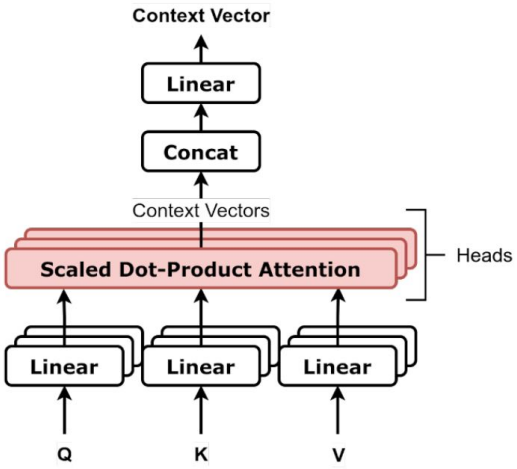


Figure 3: Multi-Head Attention

Task 1:

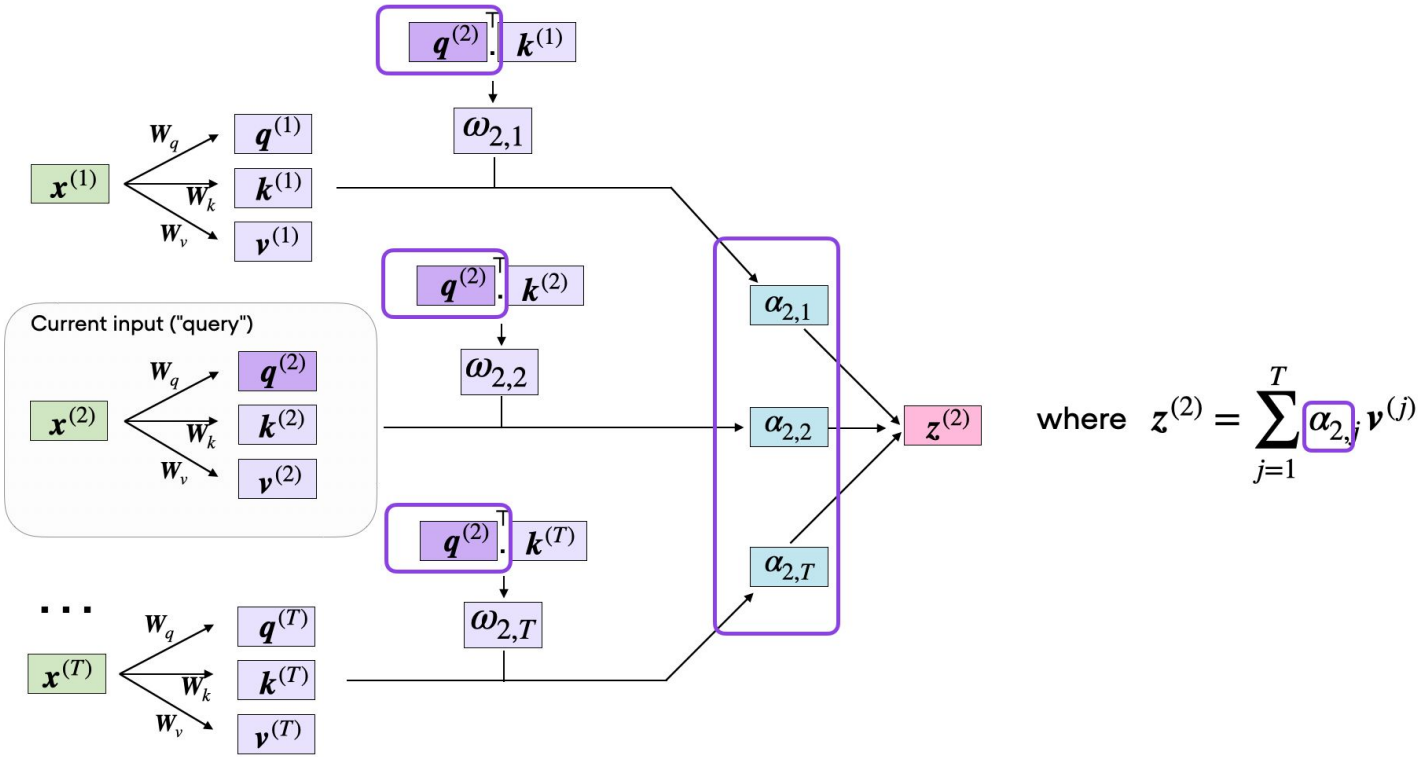
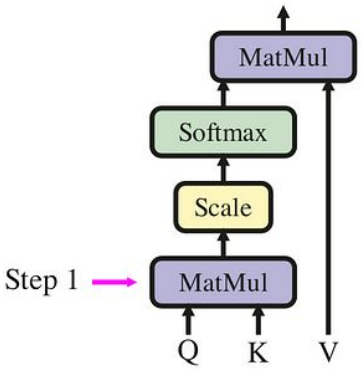
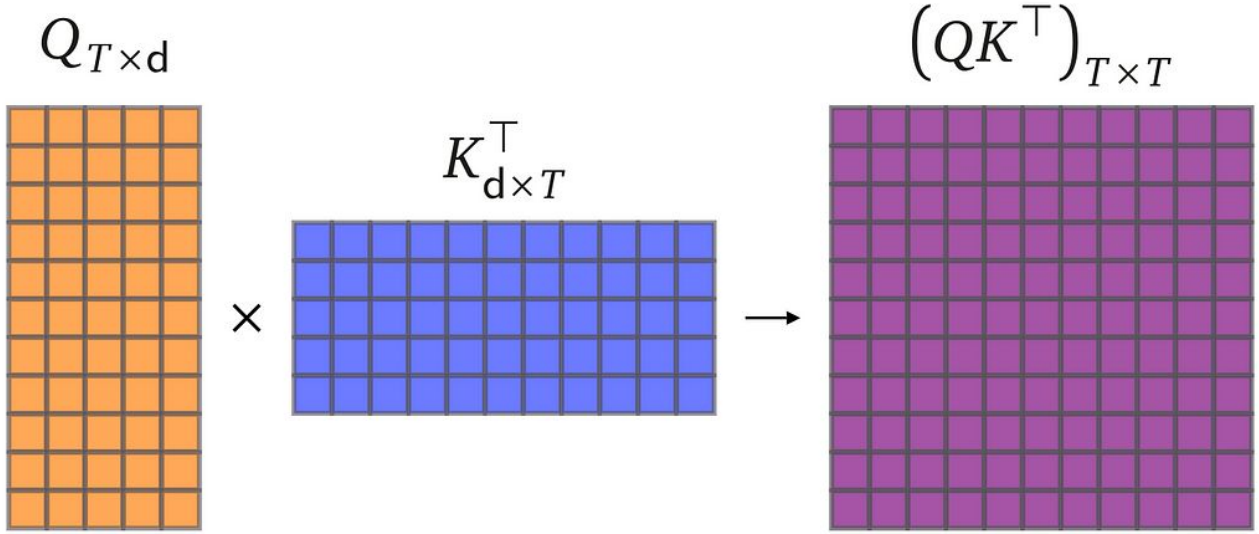


Image from: https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html

Task 1:

Q, K, V Attention in Transformers:



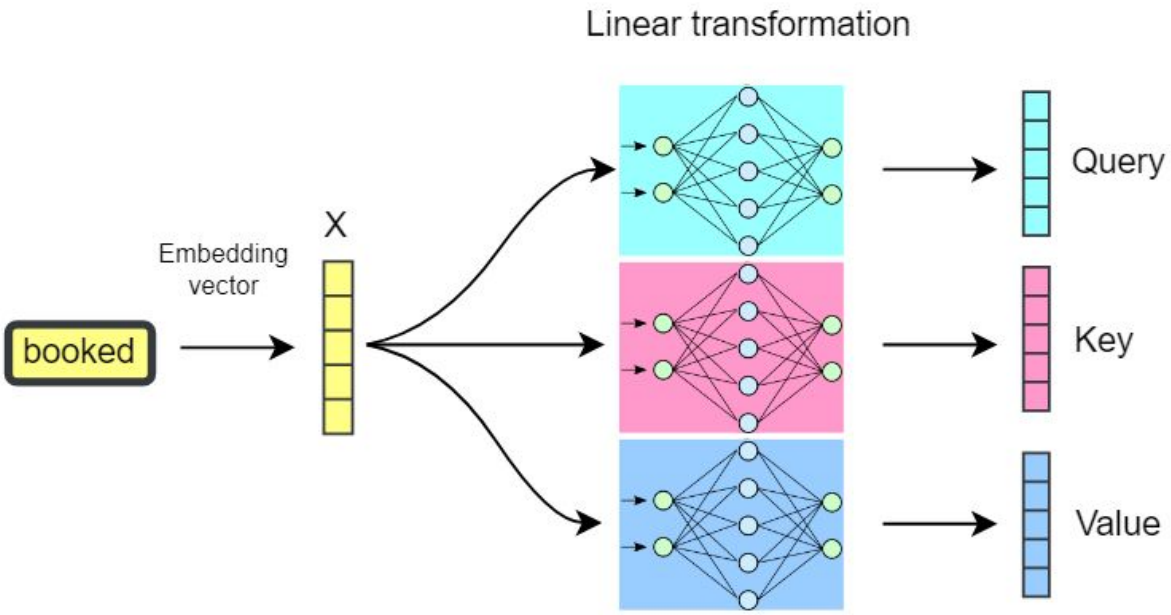


Task 1:

How do we get Query, Key and Value from a token?

Task 1:

How do we get Query, Key and Value from a token?



Task 1:

Linear Layer

- Implement a generic Linear class to handle an input A of shape *(*, in_features)*
  - **Flatten** the input tensor to 2D of shape *(batch_size, in_features)*
  - Perform the **affine** transformation
  - **Unflatten** back to original dimensions

- You can use your HW1 code as reference!

We **strongly** recommend that you incrementally test your approach to a subsection before moving on

Task 1:

Linear Layer

- Backward pass - the gradient of the loss w.r.t. the output ***Z*** will be of shape *(*, out_features)*
  - Reshape the 2D gradient of shape *(batch_size, out_features)*
  - Reshape the stored input tensor to shape *(batch_size, in_features)*
  - Compute the gradient of the loss with respect to the input
  - Finally, reshape back to the original input size

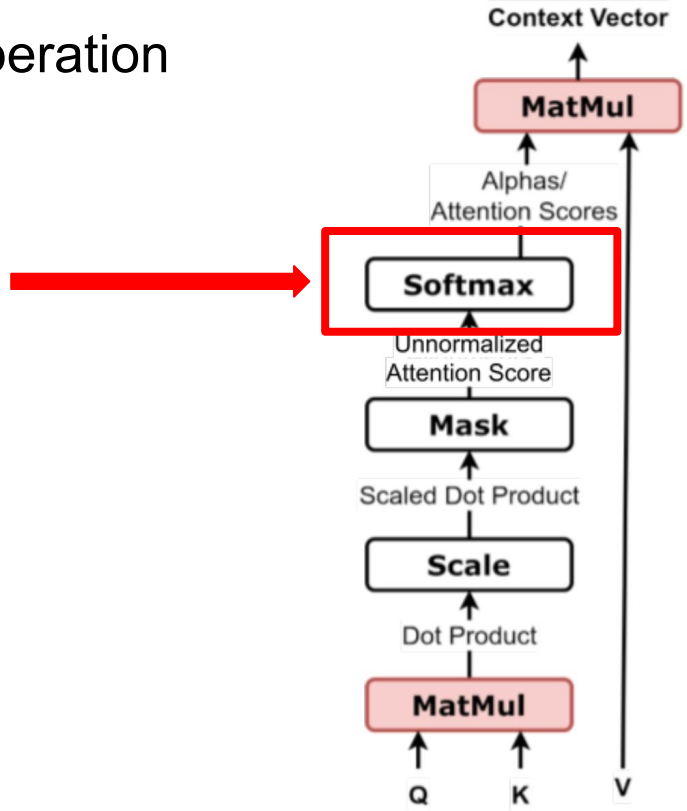
$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial Z} \cdot W$$

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial Z}\right)^T \cdot A$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^{B} \frac{\partial L}{\partial Z_i}$$

Task 1:

Softmax Operation



Task 1:

Softmax Operation

- Implement a generic Softmax class to handle an input ***Z*** of shape *(C,)* whose m^{th} element is z_m
- Softmax will return a vector ***A*** with shape *(C,)*
    - You can use your HW1 code as reference

$$a_m = \frac{\exp(z_m)}{\sum_{k=1}^{C} \exp(z_k)}$$

Task 1:

Softmax Operation

- To compute the backward, we need to compute the Jacobian **J**
- Let's call the input values **z** and probabilities (from the softmax) **a**
- Compute the gradient by *right multiplying* the loss w.r.t. affine values with the Jacobian
- The final gradient tensor will have the same shape as the input tensor.
- Checkout the [np.moveaxis]() documentation – you might find this helpful

$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{a}} \cdot \mathbf{J}$$

Task 1:

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- A query matrix $\mathbf{Q}$ of shape $(N, ..., H, L, E)$
- A key matrix $\mathbf{K}$ of shape $(N, ..., H, S, E)$
- A value matrix $\mathbf{V}$ of shape $(N, ..., H, S, Ev)$
- Optionally, we will have a boolean mask matrix $\mathbf{M}$ of shape $(N, ..., H, L, S)$

- N is the batch size
- H is the number of attention heads for the key and value
- L is the length of the target sequence
- S is the length of the source sequence
- E is the embedding dimension
- Ev is the value dimension
- ... can be any arbitrary number of dimensions

Task 1:

Scaled Dot-Product Attention

1. Gradient with respect to V:

$$\frac{\partial L}{\partial V} = A^T \cdot \frac{\partial L}{\partial O}$$

2. Gradient with respect to attention scores A:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial O} \cdot V^T$$

3. Gradient with respect to the scaled dot-product:

$$\frac{\partial L}{\partial S} = \text{softmax_backward}\left(\frac{\partial L}{\partial A}\right)$$

4. Gradient with respect to Q:

$$\frac{\partial L}{\partial Q} = \left(\frac{\partial L}{\partial S} \cdot \frac{1}{\sqrt{d_k}}\right) \cdot K$$

5. Gradient with respect to K:

$$\frac{\partial L}{\partial K} = \left(\frac{\partial L}{\partial S} \cdot \frac{1}{\sqrt{d_k}}\right)^T \cdot Q$$

Task 1:

Multi-Headed Attention

- Instead of one large attention pass, we **split** the focus into *H* independent heads.
- Each head computes its own output **independently**.
- All head outputs are **concatenated**.
- Then passed through a final weight matrix.

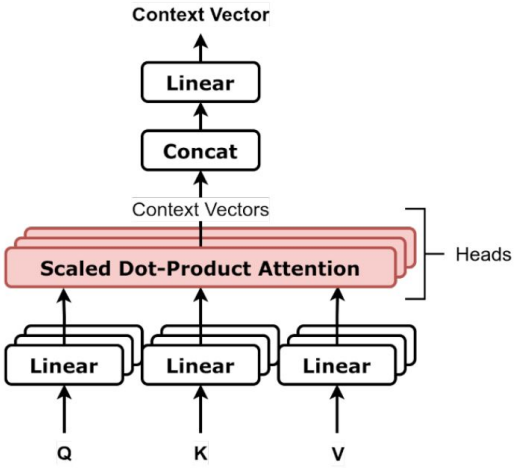
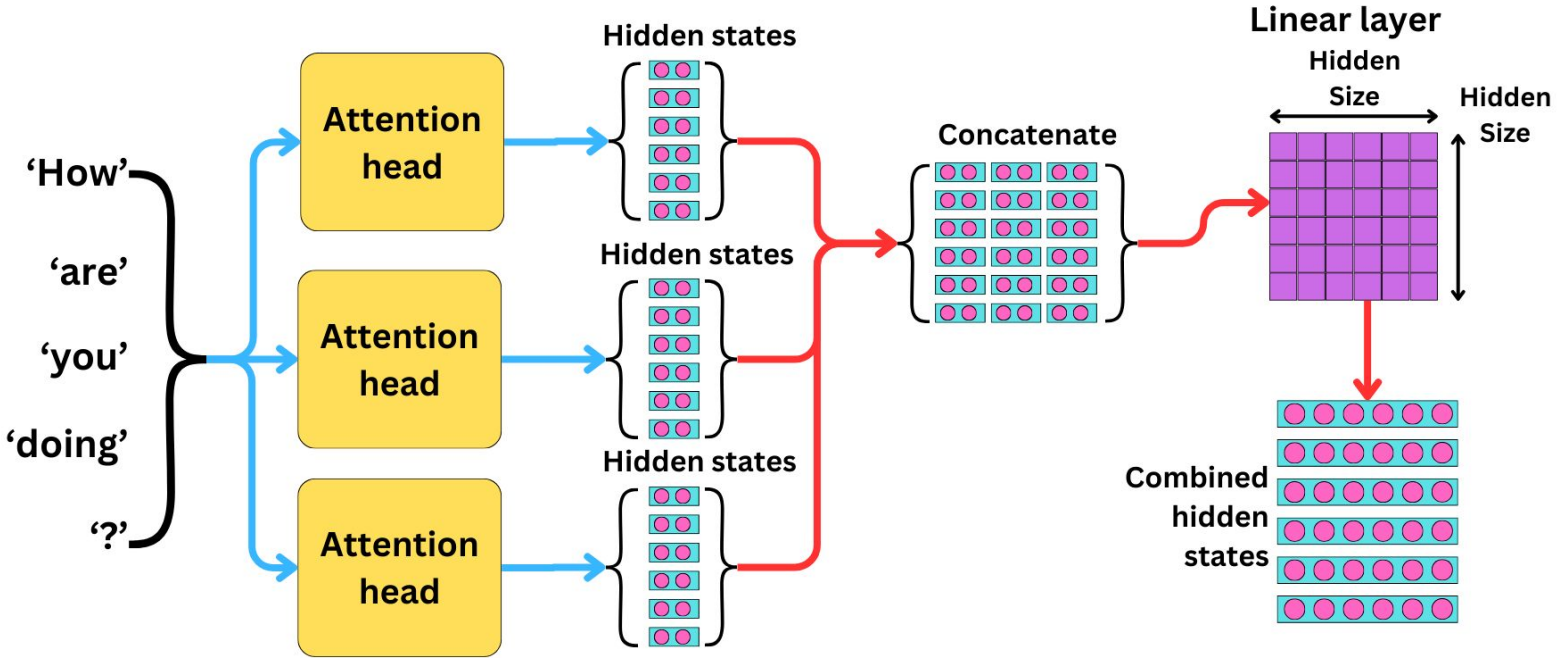


Figure 3: Multi-Head Attention

Task 1:

Think about what the critical change is to from SDPA to MHA



Task 1:

Multi-Head Attention

- Project Q, K, V matrices using Linear Layers
- Implement *_split_heads* which creates the attention heads (Think: Can we arbitrarily choose a number for num_heads?)
- Implement *_merge_masks* to merge the key-padding mask and the attention mask into a single mask
- Apply *ScaledDotProductAttention* to **each head**
- Implement *_concat_heads* to merge the output from each head
- Project the attention outputs back to the original embedding dimension

Task 1:

Multi-Head Attention (Backward)

- Backpropagate through output projection using the gradient of the loss wrt the output.
- Split gradients into multiple heads using the _split_heads method.
- Backpropagate gradients through scaled dot-product attention for each head.
- Merge heads of gradients for Q, K, V using the _concat_heads method.
- Backpropagate gradients through input projections.

Task 2:

Data-Preprocessing:

1. Load text as strings into memory.

2. Split the strings into tokens (e.g., words or characters).

3. Build a vocabulary dictionary to associate each vocabulary element with a numerical index.

4. Convert the text into sequences of numerical indices.

Task 2:

Tokenization Strategies:

- Character-level tokenization
- Subword tokenization with a vocabulary size of 1,000
- Subword tokenization with a vocabulary size of 5,000
- Subword tokenization with a vocabulary size of 10,000

Task 2:

Batching Correctly:

Load .npy files.

Tokenize the sequences using a provided tokenizer

Next:
– A shifted version with a Start-of-Sequence (SOS) token prepended,
– A golden version with an End-of-Sequence (EOS) token appended,

Provide a custom collate_fn function for batching data correctly by:

– Padding transcripts to batch-uniform lengths.

– Providing lengths for mask generation.

– Ensuring proper tensor types.

Task 2: Casual Mask

$$\text{input} = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \end{bmatrix}$$

Then the output mask will be:

$$\text{mask} = \begin{bmatrix} \text{False} & \text{True} & \text{True} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{True} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{False} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{False} & \text{False} & \text{True} \\ \text{False} & \text{False} & \text{False} & \text{False} & \text{False} \end{bmatrix}$$

Task 2: Padding Mask

$$\text{input} = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \end{bmatrix}$$

And the lengths tensor:

$$\text{lengths} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

Then the output mask will be:

$$\text{mask} = \begin{bmatrix} \text{False} & \text{False} & \text{False} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{True} & \text{True} & \text{True} \end{bmatrix}$$

Task 2: Positional Encoding

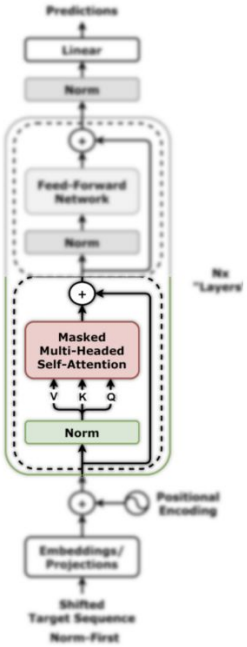
Given an input representation consisting of d-dimensional embeddings for T tokens, positional encoding generates a matrix $\mathbf{P} \in \mathbb{R}^{T \times d}$ of the same shape. The elements of $\mathbf{P}$ are computed as follows:

$$\mathbf{P}_{t,2i} = \sin\left(\frac{t}{10000^{\frac{2i}{d}}}\right), \quad \mathbf{P}_{t,2i+1} = \cos\left(\frac{t}{10000^{\frac{2i}{d}}}\right)$$

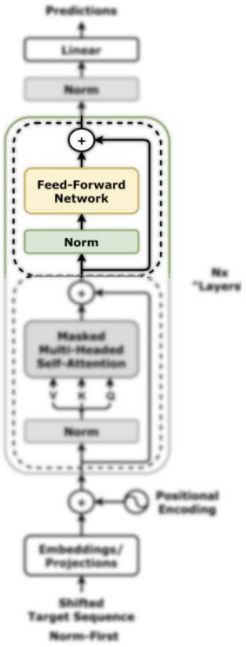
Implement the following methods of the `PositionalEncoding` class in `hw4lib/model/positional_encoding.py`:

- `create_pe_table`: Constructs the positional encoding matrix $\mathbf{P}$ of shape $(\text{max_len}, d_model)$ where:
  - `max_len`: The maximum length of input sequences.
  - `d_model`: The dimensionality of each token embedding.
- `forward`: Adds the positional encoding matrix $\mathbf{P}$ to the input embeddings before feeding them into the model.

Task 2: Transformer Sublayers

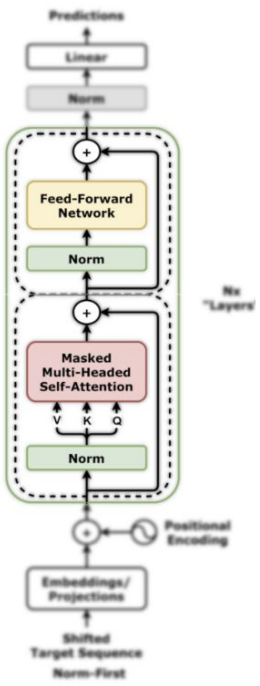


(a) Self-Attention Sublayer Architecture

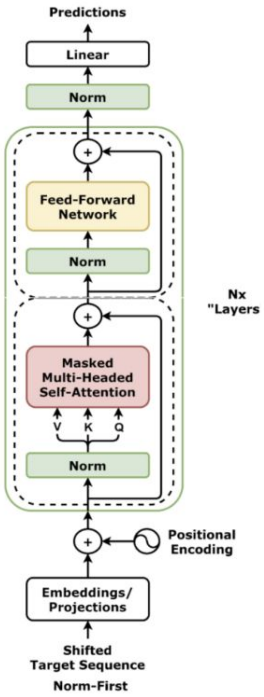


(b) FeedForward Sublayer Architecture

Task 2:



(a) Self-Attention Decoder Layer Architecture



(b) Decoder-Only Transformer Architecture

Task 2: generate_greedy in SequenceGenerator class

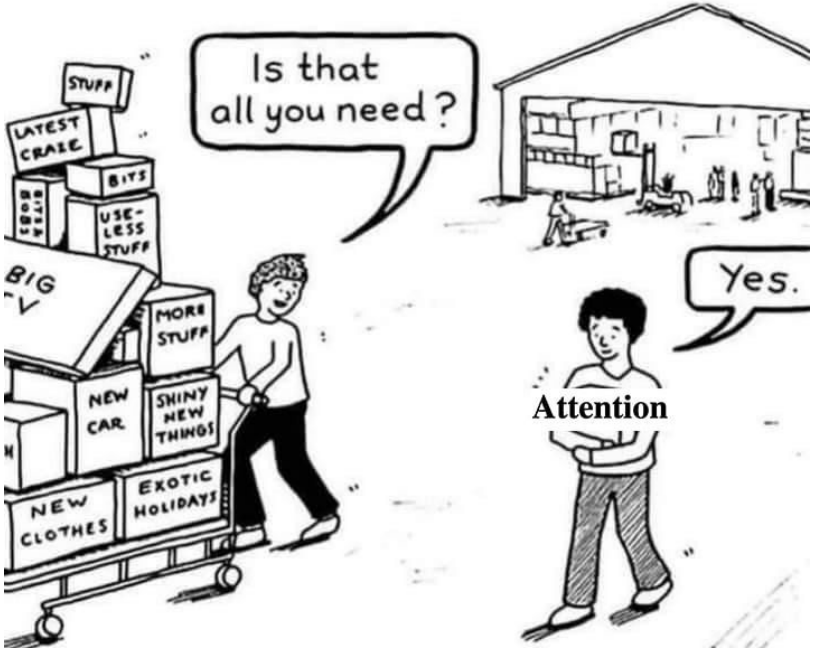
Pseudocode 1: Greedy Decoding

```
function generate_greedy(x, temperature, repeat_penalty):

    # Initialize scores and flags
    initialize scores as zeros of shape (batch_size,)
    initialize finished flags as False for each sequence

    for t in range(max_length - current_sequence_length):
        if all sequences are finished:
            break

        logits = score_fn(x)
        logits = apply_repeat_penalty(logits, x, repeat_penalty)
        logits = logits / temperature
        log_probs = log_softmax applied to raw logits

        next_tokens = tokens with highest log_probs for each sequence
        token_scores = log_probs corresponding to next_tokens

        update scores only for unfinished sequences

        append next_tokens to x

        update finished flags to True if EOS token is generated for that sequence

    return x, scores
```

Task 2: LMTrainer

Implementation Tasks:

- Initialize the loss criterion in __init__.
- Complete the training loop in _train_epoch.
- Integrate your greedy decoding implementation in generate.
- Implement the validation loop in _validate_epoch.
- Finalize the full training pipeline in train.

<u>You must achieve a per-character perplexity ≤ 3.5 in order to get points for Task 2.</u>

Please reach out if you have questions!